

Sorting algorithms (insertion, selection, bubble, cocktail, Shell, merge, quick, counting, radix, bucket sort)

Data Structures and algorithms

Attila Házy

2026

Learning goals

- Understand sorting as an operation defined by an ordering relation.
- Compare simple quadratic sorting methods and divide-and-conquer methods.
- See why some sorting algorithms are stable, adaptive, in-place, or external.
- Learn linear-time sorting methods and their assumptions.
- Study examples for insertion, selection, bubble, cocktail, Shell, merge, quick, counting, radix, and bucket sort.

Outline

- 1 Ordering relations and sorting requirements
- 2 Simple quadratic sorting methods
- 3 Cocktail and Shell sorting
- 4 Merge sort and Batcher odd-even merge
- 5 Quicksort and square-root sorting
- 6 Lower bound for comparison sorting
- 7 Linear-time sorting
- 8 External sorting
- 9 Summary

Definition

Let A be a set. A *relation* ρ on A is a subset of $A \times A$. We say that $a, b \in A$ are in relation ρ if $(a, b) \in \rho$.

Example

For $A = \mathbb{R}$, the relation $<$ contains all pairs (a, b) for which $a < b$.

Ordering relation

Definition

A relation $\preceq$ on a set A is an ordering relation if it is:

- ① **Reflexive:** $a \preceq a$ for every $a \in A$.
- ② **Antisymmetric:** if $a \preceq b$ and $b \preceq a$, then $a = b$.
- ③ **Transitive:** if $a \preceq b$ and $b \preceq c$, then $a \preceq c$.
- ④ **Total:** for any $a, b \in A$, either $a \preceq b$ or $b \preceq a$.

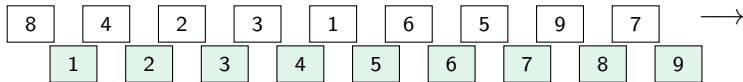
Example

The relation $\leq$ on real numbers is an ordering relation.

Definition

Sorting a sequence means rearranging its elements so that consecutive elements are in the prescribed ordering relation.

In this lecture we mostly sort integers and use the usual relation $\leq$.



Common requirements for sorting algorithms

Requirement	Meaning
In-place	The sorted result is produced in the original array using only constant extra memory.
Fast running time	The number of comparisons, swaps, and moves should be small.
Adaptive	The algorithm exploits already existing order in the input.
Stable	Records with equal keys keep their original relative order.
Pointer sorting	Only pointers to records are moved instead of complete records.
Internal / external	Internal sorting uses main memory only; external sorting uses secondary storage.
Comparison-based	The algorithm orders elements through key comparisons.
Non-comparison sort	Additional assumptions on keys make comparison unnecessary.
Online / incremental	The final result may gradually appear while processing.

Why non-optimal algorithms are still useful

- A theoretically faster algorithm may require data in a form that is expensive to prepare.
- Simple algorithms are easier to implement and debug.
- For small arrays, constant factors may dominate asymptotic behavior.
- Stability, memory usage, and adaptivity may matter more than worst-case complexity.

Main lesson

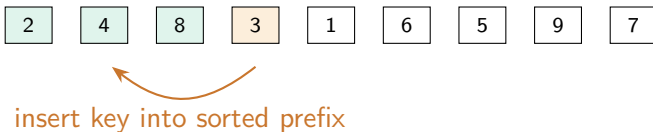
There is no universally best sorting algorithm. The best choice depends on the data and the computational environment.

Insertion sort: idea

Principle

Starting from the second element, insert each key into the already sorted prefix of the array.

At step j , the subarray $A[1..j-1]$ is already sorted. The algorithm moves $A[j]$ left until it reaches its correct position.



Insertion sort: pseudocode

```
INSERTION_SORT(A)
// Input:  A - array to be sorted
// Output: A - sorted array
FOR j ← 2 TO length[A] DO
    key ← A[j]
    // Insert key into the sorted sequence A[1..j-1]
    i ← j - 1
    WHILE i > 0 AND A[i] > key DO
        A[i+1] ← A[i]
        DEC(i)
    A[i+1] ← key
RETURN(A)
```

- Corrected duplicated line numbers from the original typesetting.
- This version is stable because equal keys are not moved past one another.

Insertion sort: example

Step	Array								
initial	8	4	2	3	1	6	5	9	7
1	4	8	2	3	1	6	5	9	7
2	2	4	8	3	1	6	5	9	7
3	2	3	4	8	1	6	5	9	7
4	1	2	3	4	8	6	5	9	7
5	1	2	3	4	6	8	5	9	7
6	1	2	3	4	5	6	8	9	7
7	1	2	3	4	5	6	8	9	7
8	1	2	3	4	5	6	7	8	9

Insertion sort: running time

- Best case: already sorted array. Only one comparison per outer iteration: $\Theta(n)$.
- Worst case: reverse sorted array. Each new key moves across the whole sorted prefix: $\Theta(n^2)$.
- Average case: $\Theta(n^2)$.

Useful properties

Insertion sort is in-place, stable, and adaptive. It is very efficient for small or nearly sorted arrays.

Selection sort: idea

Principle

Repeatedly find the minimum element in the unsorted suffix and swap it with the first element of that suffix.

After iteration i , the prefix $A[1..i]$ contains the i smallest elements in sorted order.



Selection sort: pseudocode

```
SELECTION_SORT(A)
// Input:  A - array to be sorted
// Output: A - sorted array
FOR i <- 1 TO length[A]-1 DO
    // Search minimum in A[i..length[A]]
    k <- i
    x <- A[i]
    FOR j <- i+1 TO length[A] DO
        IF A[j] < x THEN
            k <- j
            x <- A[j]
    // Swap A[i] and A[k]
    A[k] <- A[i]
    A[i] <- x
RETURN(A)
```

Selection sort: example

Pass	Minimum selected	Array after swap								
0	-	8	4	2	3	1	6	5	9	7
1	1	1	4	2	3	8	6	5	9	7
2	2	1	2	4	3	8	6	5	9	7
3	3	1	2	3	4	8	6	5	9	7
4	4	1	2	3	4	8	6	5	9	7
5	5	1	2	3	4	5	6	8	9	7

Selection sort: running time and properties

- Number of comparisons is fixed:

$$(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2} = \Theta(n^2).$$

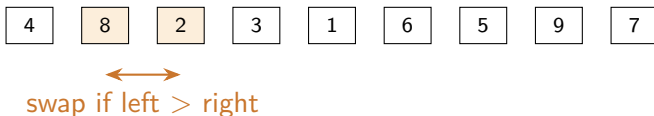
- Number of swaps is at most $n - 1$.
- In-place: yes.
- Stable: not in its standard swap-based version.
- Adaptive: no; it does the same comparisons even if the input is already sorted.

Bubble sort: idea

Principle

Compare adjacent elements and swap them if they are in the wrong order. Repeat passes until no swap is needed.

Large elements “bubble” toward the end of the array.



Bubble sort: corrected pseudocode

```
BUBBLE_SORT(A)
// Input:  A - array to be sorted
// Output: A - sorted array
j <- 2
REPEAT
    noSwap <- TRUE
    FOR i <- length[A] DOWNTO j DO
        IF A[i] < A[i-1] THEN
            SWAP(A[i], A[i-1])
            noSwap <- FALSE
    INC(j)
UNTIL noSwap
RETURN(A)
```

- The Boolean variable is written consistently as noSwap.
- The early stop makes the algorithm adaptive on already sorted input.

Bubble sort: example

Input: [5, 1, 4, 2, 8].

Pass	Array after pass				
initial	5	1	4	2	8
1	1	4	2	5	8
2	1	2	4	5	8
3	no swap, stop				

- Worst case: $\Theta(n^2)$ comparisons and swaps.
- Best case with early stop: $\Theta(n)$.
- Stable and in-place.

Principle

Cocktail sort is a bidirectional version of bubble sort. One pass moves a large element to the right, the next pass moves a small element to the left.

- It improves the slow movement of small elements in ordinary bubble sort.
- It is also called *shaker sort* or *bidirectional bubble sort*.
- It remains a quadratic-time sorting algorithm.

Cocktail sort: pseudocode

```
COCKTAIL_SORT(A)
left <- 1
right <- length[A]
REPEAT
    swapped <- FALSE
    FOR i <- left TO right-1 DO
        IF A[i] > A[i+1] THEN
            SWAP(A[i], A[i+1])
            swapped <- TRUE
    DEC(right)
    IF NOT swapped THEN RETURN(A)
    swapped <- FALSE
    FOR i <- right DOWNTO left+1 DO
        IF A[i-1] > A[i] THEN
            SWAP(A[i-1], A[i])
            swapped <- TRUE
    INC(left)
UNTIL NOT swapped
RETURN(A)
```

Cocktail sort: example

Input: [5, 1, 4, 2, 8, 0, 3].

Stage	Array						
initial	5	1	4	2	8	0	3
left-to-right	1	4	2	5	0	3	8
right-to-left	0	1	4	2	5	3	8
next left-to-right	0	1	2	4	3	5	8
next right-to-left	0	1	2	3	4	5	8

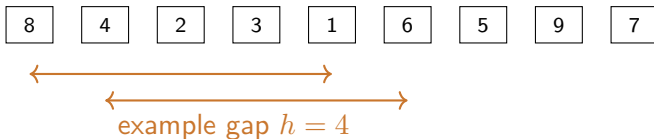
Cocktail sort: properties

- Worst-case running time: $\Theta(n^2)$.
- Best-case running time with early stopping: $\Theta(n)$.
- In-place: yes.
- Stable: yes, if equal elements are not swapped.
- Compared with bubble sort, it can handle “turtles” (small elements near the end) faster.

Shell sort: idea

Principle

Shell sort accelerates insertion sort by comparing and moving elements that are far apart. The gap is then gradually reduced until it becomes 1.



For each gap, the algorithm performs insertion sort on subsequences separated by that gap.

Shell sort: gap sequences

Use a decreasing sequence of increments:

$$h_1 > h_2 > \dots > h_t = 1.$$

Common examples:

- Shell's original sequence: $\dots, 32, 16, 8, 4, 2, 1$.
- Knuth sequence: $\dots, 121, 40, 13, 4, 1$.
- Hibbard sequence: $\dots, 31, 15, 7, 3, 1$.

Correction and clarification

The running time strongly depends on the chosen gap sequence. Shell sort is not simply characterized by one universal bound.

Shell sort: pseudocode

```
SHELL_SORT(A, h[1..t])
// h[1] > h[2] > ... > h[t] = 1
FOR s <- 1 TO t DO
    gap <- h[s]
    FOR j <- gap+1 TO length[A] DO
        record <- A[j]
        key <- key[record]
        i <- j - gap
        WHILE i > 0 AND key < key[A[i]] DO
            A[i+gap] <- A[i]
            i <- i - gap
        A[i+gap] <- record
RETURN(A)
```

- Corrected the notation: if $A[j]$ is a record, compare by $\text{key}[A[j]]$; for numeric arrays, simply use $A[j]$.

Shell sort: example with gaps 4, 2, 1

Input: [8, 4, 2, 3, 1, 6, 5, 9, 7].

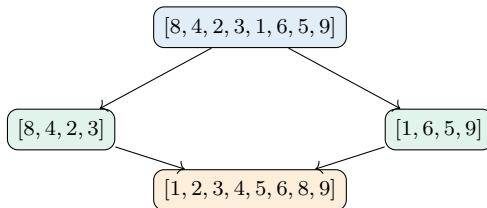
Gap	Result after gap-based insertion passes								
initial	8	4	2	3	1	6	5	9	7
4	1	4	2	3	7	6	5	9	8
2	1	3	2	4	5	6	7	9	8
1	1	2	3	4	5	6	7	8	9

- In-place: yes.
- Stable: generally no.
- Complexity depends on gaps; often much faster than insertion sort in practice.

Merge sort: idea

Divide and conquer

- 1 Divide the array into two halves.
- 2 Recursively sort both halves.
- 3 Merge the two sorted halves.



Merge operation

```
MERGE(A, p, q, r)
// A[p..q] and A[q+1..r] are sorted
// Copy the smaller current element into a temporary array
// Continue until one part is empty
// Copy the remaining elements
// Copy the temporary array back into A[p..r]
```

- Merging n total elements takes $\Theta(n)$ time.
- Standard merge sort needs linear extra memory.
- Merge sort is stable if equal keys are taken from the left half first.

Merge sort: pseudocode

```
MERGE_SORT(A, p, r)
// Sort subarray A[p..r]
IF p < r THEN
    q <- floor((p+r)/2)
    MERGE_SORT(A, p, q)
    MERGE_SORT(A, q+1, r)
    MERGE(A, p, q, r)
RETURN(A)
```

- Corrected the missing formula for the middle index:

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor.$$

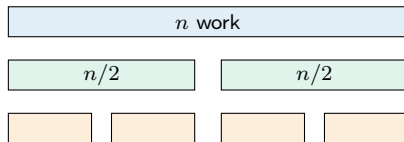
- To sort the full array: `MERGE_SORT(A, 1, length[A])`.

Merge sort: running time

$$T(n) = 2T(n/2) + \Theta(n),$$
$$T(1) = \Theta(1).$$

By the Master Theorem:

$$T(n) = \Theta(n \log n).$$



$\log_2 n$ levels, $\Theta(n)$ work per level

Batcher odd-even merge: what it is

Purpose

Batcher's odd-even merge is a method for merging two sorted sequences in a way that is especially suitable for parallel implementation.

Correction

The standard odd-even merge splits both input sequences into odd-indexed and even-indexed subsequences, merges odds with odds and evens with evens, then performs compare-exchange operations between neighboring positions. It is not an independent sorting method by itself.

Batcher odd-even merge: structure

Let

$$A = (a_1, a_2, \dots, a_n), \quad B = (b_1, b_2, \dots, b_m)$$

be sorted. Split:

$$A_o = (a_1, a_3, \dots), \quad A_e = (a_2, a_4, \dots),$$

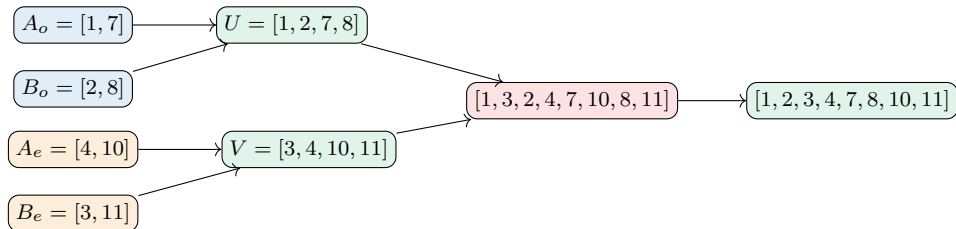
$$B_o = (b_1, b_3, \dots), \quad B_e = (b_2, b_4, \dots).$$

Then:

- 1 Merge A_o with B_o to get U .
- 2 Merge A_e with B_e to get V .
- 3 Interleave U and V .
- 4 Compare-exchange adjacent pairs to finish the merge.

Batcher odd-even merge: example

$A = [1, 4, 7, 10], \quad B = [2, 3, 8, 11].$

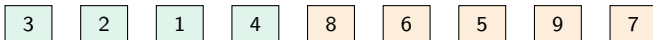


Final adjacent compare-exchange fixes the interleaved sequence.

Quicksort: idea

Divide and conquer

- 1 Partition $A[p..r]$ into two nonempty parts $A[p..q]$ and $A[q + 1..r]$ so that every element in the first part is at most every element in the second part.
- 2 Recursively sort both parts.
- 3 No merge is needed.



Quicksort: pseudocode

```
QUICKSORT(A, p, r)
// Sort subarray A[p..r]
IF p < r THEN
    PARTITION(A, p, r, A[p], q)
    QUICKSORT(A, p, q)
    QUICKSORT(A, q+1, r)
RETURN(A)
```

- The partition procedure may be the Hoare-style partition used earlier.
- Pivot choice is crucial for performance.
- Quicksort is in-place in common implementations, but not stable by default.

Quicksort: running time

- Worst case: each partition has sizes 0 and $n - 1$:

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

- Best case: balanced halves:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

- Average case: $\Theta(n \log n)$.

Practical note

Randomized pivot selection or median-of-three pivoting greatly reduces the chance of worst-case behavior.

Quicksort: example partition

Use pivot 4 on [4, 8, 2, 3, 1, 6, 5, 9, 7].

Stage	Array								
initial	4	8	2	3	1	6	5	9	7
partitioned	1	3	2	4	8	6	5	9	7
recursive sort	1	2	3	4	5	6	7	8	9

Square-root sorting

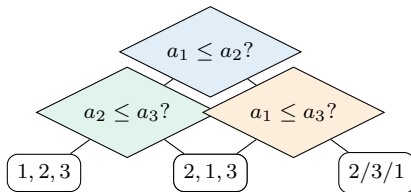
Idea

Divide the n elements into about $\sqrt{n}$ groups, each containing about $\sqrt{n}$ elements. Extract the minimum from each subgroup into a main group, repeatedly output the smallest main-group element, and refill from its original subgroup.

- The method resembles repeated multiway selection.
- The described form has comparison count about $\Theta(n\sqrt{n})$.
- A multilevel version can reduce the exponent but remains mainly of theoretical or historical interest compared with heap sort or merge sort.

Decision tree model

- A comparison-based sorting algorithm can be modeled by a binary decision tree.
- Internal nodes are comparisons such as $a_i \leq a_j$.
- Leaves represent possible output permutations.
- To sort all inputs, the tree needs at least $n!$ leaves.



Stirling formula

Theorem

For $n = 3, 4, 5, \dots$,

$$\sqrt{2\pi} n^{n+1/2} e^{-n} < n! < e n^{n+1/2} e^{-n}.$$

A simpler asymptotic form is:

$$n! = \Theta\left(\sqrt{n} \left(\frac{n}{e}\right)^n\right).$$

This is used to estimate $\log_2(n!)$.

Lower bound for comparison sorting

Theorem

Any comparison-based sorting algorithm needs height at least

$$\lceil \log_2(n!) \rceil$$

in its decision tree.

Proof idea.

A binary tree of height h has at most 2^h leaves. Sorting n distinct elements has $n!$ possible permutations, so $2^h \geq n!$. Therefore $h \geq \log_2(n!) = \Omega(n \log n)$ by Stirling's formula. $\square$

Important conclusion

No comparison-based sorting algorithm can have worst-case running time better than $\Omega(n \log n)$.

- Merge sort achieves $\Theta(n \log n)$ worst-case time.
- Quicksort achieves $\Theta(n \log n)$ on average but $\Theta(n^2)$ in the worst case.
- Linear-time sorting is possible only when we use extra information about the keys.

Counting sort: assumptions

Input assumption

Counting sort sorts integers from a small range, usually $1, 2, \dots, k$ or $0, 1, \dots, k$.

It uses:

- input array $A[1..n]$,
- output array $B[1..n]$,
- helper array $C[1..k]$.

Running time:

$$\Theta(n + k).$$

If $k = O(n)$, then the running time is linear in n .

Counting sort: corrected pseudocode

```
COUNTING_SORT(A, k, B)
FOR i ← 1 TO k DO
    C[i] ← 0
FOR j ← 1 TO length[A] DO
    INC(C[A[j]])
// C[i] is the number of occurrences of i
FOR i ← 2 TO k DO
    C[i] ← C[i] + C[i-1]
// C[i] is the number of elements ≤ i
FOR j ← length[A] DOWNTO 1 DO
    B[C[A[j]]] ← A[j]
    DEC(C[A[j]])
RETURN(B)
```

Correction

The cumulative counting step must be $C[i] \leftarrow C[i] + C[i - 1]$, not $C[i] + C[i + 1]$.

Counting sort: example

Input: $A = [4, 2, 2, 8, 3, 3, 1]$, $k = 8$.

i	1	2	3	4	5	6	7	8
counts $C[i]$	1	2	2	1	0	0	0	1
cumulative $C[i]$	1	3	5	6	6	6	6	7

Output:

$$B = [1, 2, 2, 3, 3, 4, 8].$$

- The final loop goes right-to-left, making the algorithm stable.
- Stability is essential when counting sort is used inside radix sort.

Idea

Sort fixed-length keys digit by digit using a stable sorting method for each digit.

For least-significant-digit radix sort:

- 1 Sort by the last digit.
- 2 Then by the previous digit.
- 3 Continue until the first digit.

If there are d digits, n records, and k possible digit values, then with counting sort:

$$T(n) = \Theta(d(n + k)).$$

Radix sort: pseudocode

```
RADIX_SORT(A)
// All keys have d digits
FOR i <- d DOWNTO 1 DO
    stably sort A according to digit i
RETURN(A)
```

- The stable subroutine is often counting sort.
- It works well for dates, fixed-length strings, and fixed-width numeric keys.

Radix sort: example

Sort [329, 457, 657, 839, 436, 720, 355].

Digit pass	Array after stable sorting by that digit
ones	[720, 355, 436, 457, 657, 329, 839]
tens	[720, 329, 436, 839, 355, 457, 657]
hundreds	[329, 355, 436, 457, 657, 720, 839]

The stability of each digit sort preserves the ordering established by previous less significant digits.

Bucket sort

Input assumption

Bucket sort assumes the input is drawn uniformly from the interval $[0, 1)$.

Procedure:

- 1 Create n buckets $B[0], \dots, B[n-1]$.
- 2 Put $A[i]$ into bucket $\lfloor nA[i] \rfloor$.
- 3 Sort each bucket, usually by insertion sort.
- 4 Concatenate the buckets.

Expected running time under the uniformity assumption:

$$\Theta(n).$$

Bucket sort: pseudocode

```
BUCKET_SORT(A, L)
n ← length[A]
FOR i ← 0 TO n-1 DO
    B[i] ← empty list
FOR i ← 1 TO n DO
    insert A[i] into B[floor(n*A[i])]
FOR i ← 0 TO n-1 DO
    sort B[i] by insertion sort
L ← concatenate B[0], B[1], ..., B[n-1]
RETURN(L)
```

- The original text says to insert into a bucket list; here the precise bucket index is written explicitly.

Bucket sort: example

Input:

[0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68].

With $n = 10$ buckets:

Bucket	Elements
1	0.12, 0.17
2	0.21, 0.23, 0.26
3	0.39
6	0.68
7	0.72, 0.78
9	0.94

Concatenating sorted buckets gives the final sorted sequence.

Context

External sorting is used when the data do not fit into main memory. Access and movement time on secondary storage dominate the cost.

Merge-based methods are the most important external sorting techniques.

Definition

A *run of length k* in a file is a block of k neighboring records whose keys are already sorted.

- Initially, $k = 1$ always works.
- If natural runs already exist, using them can reduce the number of passes.

Balanced two-way external merge sort

- 1 Split the input file into two auxiliary files containing runs of length k .
- 2 Merge corresponding runs into an output file. The new run length is $2k$.
- 3 Repeat, doubling k until the whole file is sorted.

If there are n records:

- each pass moves $\Theta(n)$ records,
- the number of passes is at most $\lceil \log_2 n \rceil$,
- total movement cost is $\Theta(n \log n)$.

Improving external sorting

- **Natural runs:** use already sorted runs in the input file.
- **More auxiliary files:** split into several files and merge multiple runs at a time.
- **Polyphase merge:** the file that becomes empty is immediately reused as the next output file.

Practical remark

External sorting performance depends more on I/O organization than on CPU comparisons.

Polyphase merge: illustrative table

The table entries show the number of runs; parentheses indicate run length.

File / pass	1	2	3	4	5	6
1	1(1)	0	1(3)	0	1(5)	0
2	5(1)	4(1)	3(1)	2(1)	1(1)	0
3	0	1(2)	0	1(4)	0	1(6)

This example shows that polyphase merging can be inefficient with unfavorable initial run distributions.

Comparison of sorting methods

Algorithm	Best / average	Worst	In-place	Stable
Insertion sort	$\Theta(n) / \Theta(n^2)$	$\Theta(n^2)$	yes	yes
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	yes	no
Bubble sort	$\Theta(n) / \Theta(n^2)$	$\Theta(n^2)$	yes	yes
Cocktail sort	$\Theta(n) / \Theta(n^2)$	$\Theta(n^2)$	yes	yes
Shell sort	gap-dependent	gap-dependent	yes	no
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	no	yes
Quicksort	$\Theta(n \log n)$ avg.	$\Theta(n^2)$	usually	no
Counting sort	$\Theta(n + k)$	$\Theta(n + k)$	no	yes
Radix sort	$\Theta(d(n + k))$	$\Theta(d(n + k))$	no	yes, if subroutine stable
Bucket sort	$\Theta(n)$ expected	$\Theta(n^2)$	no	depends

Key takeaways

- Quadratic algorithms are simple and often useful for small or nearly sorted inputs.
- Merge sort gives guaranteed $\Theta(n \log n)$ time but needs extra memory.
- Quicksort is usually very fast but has a quadratic worst case without care.
- Counting, radix, and bucket sort avoid the comparison lower bound by using assumptions about keys.
- External sorting is mainly an I/O problem, and merge-based methods dominate.

Corrections and additions included

- Added cocktail sort after bubble sort and before Shell sort.
- Corrected counting sort cumulative count step: $C[i] \leftarrow C[i] + C[i - 1]$.
- Added missing midpoint in merge sort: $q = \lfloor (p + r)/2 \rfloor$.
- Clarified Shell sort complexity depends on the gap sequence.
- Clarified standard Batchmer odd-even merge splitting and final compare-exchange step.
- Added worked examples throughout the presentation.